Post-Quantum Cryptography Web Application

Team Members

- *Abdul Rehman*
- *Hammad Bakhtiar*
- GitHub Repository: https://github.com/rehman-batt/Info_Project

---

1. Introduction

The increasing potential of quantum computers poses a serious threat to classical public-key cryptographic systems such as RSA and ECC. This has led to the development of post-quantum cryptographic (PQC) algorithms that are resistant to attacks from quantum computers. The aim of this project is to design and develop a secure web application using Flask that demonstrates the functionality of one such PQC algorithm. The application supports public/private key generation, encryption, and decryption of textual messages using Kyber512, a lattice-based post-quantum encryption scheme.

2. Objectives

- Understand the need for post-quantum cryptography in the context of quantum computing threats.
- Implement basic PQC operations (key generation, encryption, decryption) in a Python Flask application.
- Develop a clean and interactive frontend interface to allow user interaction.
- Ensure the security of the application through appropriate Flask security practices.

3. Tools and Technologies

- **Programming Language:** Python 3.x
- **Web Framework:** Flask
- **Cryptography Library:** pqcrypto (Kyber512 implementation)

- **Frontend:** HTML, CSS, JavaScript
- **Security:** Flask-Talisman
- **Version Control:** Git and GitHub
- **Deployment Platform (Optional):** Vercel

4. Post-Quantum Cryptographic Algorithm

This project implements the **Kyber512** encryption scheme from the `kyber_py.kyber` module. Kyber is one of the selected algorithms from the NIST Post-Quantum Cryptography Standardization project. It is based on Module-Lattice structures and offers fast key encapsulation mechanisms (KEM) along with IND-CCA2 security.

Key Features of Kyber512:

- Based on lattice problems believed to be hard for quantum computers.
- Efficient and scalable for constrained environments.
- Generates public/private key pairs and allows encapsulation of shared secrets.

5. System Architecture and Features

5.1 Functional Features

- **Key Generation:**
  Users can generate a public/private key pair using Kyber512 and view the output.
- **Message Encryption:**
  A user-provided message is encrypted using the public key and encoded for display.
- **Message Decryption:**
  The encrypted ciphertext is decrypted using the private key, revealing the original message.

5.2 Frontend Design

- Developed using basic HTML, CSS, and JavaScript.
- Clean interface with responsive layout.
- Dedicated sections for entering inputs and displaying outputs.

- Scrollable boxes for displaying keys and ciphertexts.
- Client-side validation for user input fields.

5.3 Backend Design

- Flask routes handle form submissions and call appropriate cryptographic functions.
- Separate functions for key generation, encryption, and decryption are implemented in Python.
- Proper exception handling ensures graceful failure with informative messages.

5.4 Security Considerations

- **Flask-Talisman** is used to enforce secure HTTP headers such as Content Security Policy and HSTS.
- Error messages are sanitized to avoid information leakage.
- No stack traces or sensitive server-side information are shown to the user.
- User input is validated before processing.

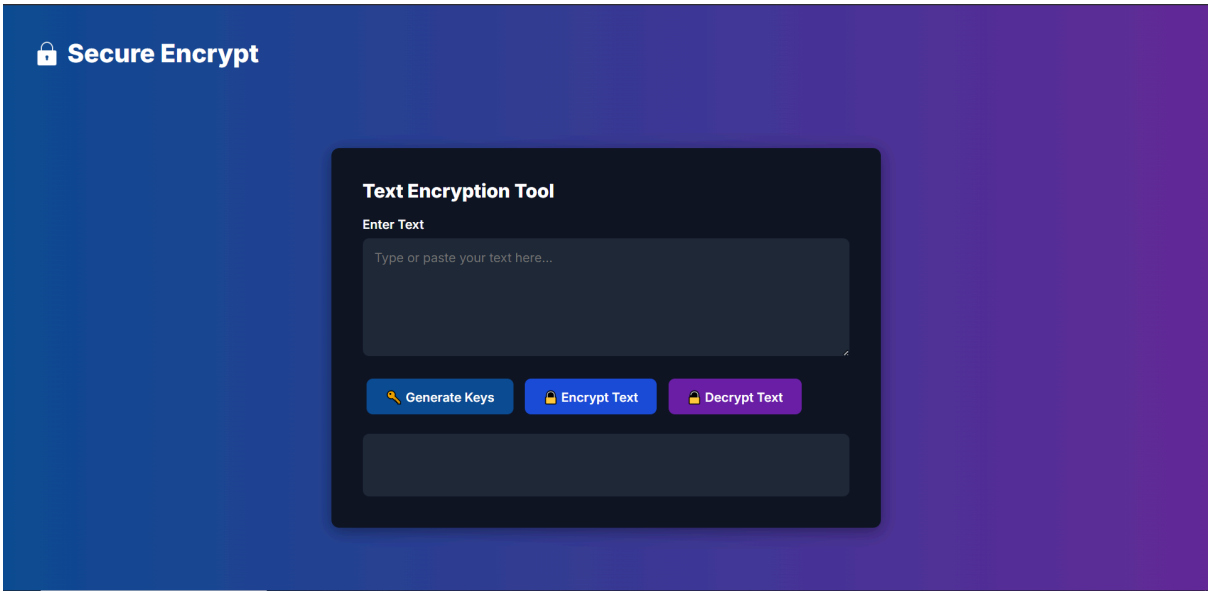
7. Implementation Process

1. **Environment Setup:**
   A virtual environment was created, and dependencies such as Flask, pqcrypto, and Flask-Talisman were installed.
2. **Backend Development:**
   - Flask routes were implemented to handle user input and invoke cryptographic functions.
   - PQC logic was implemented using `kyber_py.kyber`.
3. **Frontend Development:**
   - A single-page form was designed for user interaction.
   - Keys and results were displayed using scrollable output areas.
4. **Security Enhancements:**

- Flask-Talisman was configured.
- All errors were handled gracefully without exposing internal server details.

5. **Testing and Validation:**
    - The application was tested with various inputs to ensure correct encryption/decryption behavior.
    - Error handling was verified for malformed keys and ciphertext.

8. Screenshots

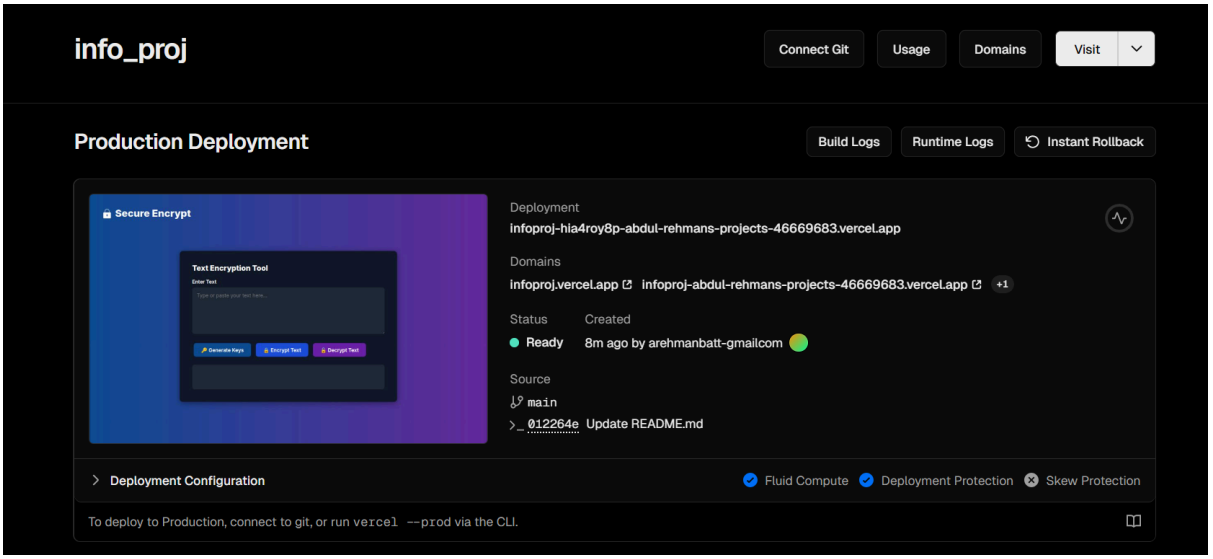


---

9. Deployment

An optional deployment was done using the Vercel platform. The application can be accessed through the following URL:

https://infoproj-b84yjtxd9-abdul-rehmans-projects-46669683.vercel.app/



11. Conclusion

This project successfully demonstrates a basic implementation of post-quantum cryptography through a secure and user-friendly web application. It showcases how real-world systems can transition toward quantum-safe cryptographic solutions using lattice-based algorithms like Kyber512. The project also emphasizes the importance of frontend/backend integration, secure web development, and emerging cryptographic standards.